5 Introduction to Data Layers

5.1 Objectives

The objective of the course material is to enable the trainee to understand the following:

- Using servlets to perform business logic and database operations
- Segregating the database operational logic as a separate layer
- Introducing service layer for transmitting information between servlets and data layer

5.2 Audience

This document is designed for Java EE Learners and programmers with a need to understand the Servlets in detail with regards to the usage of database operations. The trainees who have understood the servlet programming and overriding of servlet methods are the intended audience.

5.3 Prerequisites

Before proceeding with this document you should have a fair knowledge about the Servlet Programming and JDBC

5.4 Course Content

5.4.1 Introduction

The dynamic content to be displayed for Web applications mostly come from databases. The information residing in the databases are retrieved to be displayed in the web applications. The input as received from the user in the web applications are also updated to the databases. The request and response processing in the mentioned scenarios are done with the use of servlets that act as a controller. The power of the servlet is best visualized by its ability to handle multiple operations in terms of request handling, implementing the necessary business logic and the construction of the final response that has to be rendered back to the client (the originator of the request)

5.4.2 Servlet with JDBC implementation

A servlet is a simple Java class extending the HttpServlet (as our requests and responses would be transmitted over HTTP). The servlet class has methods doGet() and doPost() that are overridden based on the method attribute as declared in the form element of the User Interface. Consider an example of a user login page having fields, username and password. If the user is new to the application, the user needs to register to proceed to login and use the subsequent features of the application.

UI pages: Login.html / Register.html / Welcome.html
Servlet: LoginServlet / RegisterServlet
Database table: Users with structure that is covered in the database design section

The HTML code would resemble the below:

Login.html

```
<html>
<head>
<title>Login Page</title>
</head>
<body>
	<h3>Login with username and password</h3>
	<formaction="Login"method="post">
		<strong>User Name</strong>:<inputtype="text"name="username"><br>
		<strong>Password</strong>:<inputtype="password"name="password"><br>
		<inputtype="submit"value="Login">
	</form>
	<br> If you are new user, please
	<ahref="Register.html">Register</a>.
</body>
</html>
```

The Login.html displays two fields to capture username and password that are required for Login.

Register.html

```
<html>
<head>
<title>Register Page</title>
</head>
<body>
	<h3>Provide all the fields for registration.</h3>
	<formaction="Register"method="post">
		<strong>User Name</strong>:<inputtype="text"name="username"><br>
		<strong>Password</strong>:<inputtype="password"name="password"><br>
		<strong>Name</strong>:<inputtype="text"name="name"><br>
		<strong>Country</strong>:<inputtype="text"name="country"><br>
		<inputtype="submit"value="Register">
	</form>
	<br> If you are registered user, please
	<ahref="Login.html">Login</a> .
</body>
</html>
```

Here the Register.html captures the information for a new user to signup. The information of the new user is captured via the fields as displayed in this page.

Welcome.html

The Welcome.html file displays a Welcome message to the user who is successfully logged in using the Username and Password.

```
<html>
<head>
<title>Welcome Page</title>
</head>
<body>
       <h3>Welcome to Servlets Programming</h3>
       <form>
              <h4>Dear User, You have successfully logged in to our
                     applications</h4>
       </form>
</body>
</html>
```

Database Design:

Create a table Users to manage the users who sign up for the webapplication. Sample script for the same is as below:

Create table Users (
Username varchar2(20) not null,
Password varchar2(20) not null,
Name varchar2(50),
Country varchar2(20)
);

LoginServlet.java:

The LoginServlet has the doPost() method overridden as the method invocation has been set to 'post' in the Login.html. This business logic that would be performed here is to validate the user credentials (the username and the password) with the data available in the Users table of the database.

```
package com.tcs.ilp.servlet;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class LoginServlet extends HttpServlet {

	/**
	 *
	 */
	private static final long serialVersionUID = 1L;

	protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
		String uname = request.getParameter("username");
		String password = request.getParameter("password");
		String errorMsg = null;
		if(uname == null || uname.equals("")){
			errorMsg ="User Name can't be null or empty";
		}
		if(password == null || password.equals("")){
			errorMsg = "Password can't be null or empty";
		}

		if(errorMsg != null){
			RequestDispatcher rd =
getServletContext().getRequestDispatcher("/login.html");
			PrintWriter out= response.getWriter();
			out.println("<font color=red>"+errorMsg+"</font>");
```

```
                        rd.include(request, response);
                }else{

                String serverName = "172.24.137.30";
                String portNumber = "1521";
                String sid = "orcl";
                String url = "jdbc:oracle:thin:@" + serverName + ":" + portNumber + ":" +
sid;
                String dbUsername =  "e137852";
                String dbPassword = "cJRxepdfp";
                PreparedStatement ps = null;
                Connection conn =null;
                ResultSet rs = null;
                try {
// load the JDBC-ODBC Bridge driver
                        Class.forName("oracle.jdbc.driver.OracleDriver");

                        // connect to db using DriverManager
                        System.out.println("The Jdbc URL is "+url);
                        conn =DriverManager.getConnection( url, dbUsername, dbPassword );
                        ps = conn.prepareStatement("select name, username ,country from
Users where username=? and password=? limit 1");
                        ps.setString(1, uname);
                        ps.setString(2, password);
                        rs = ps.executeQuery();

                        if(rs != null&& rs.next()){
                                // Print welcome message to user
                                RequestDispatcher rd =
getServletContext().getRequestDispatcher("/Welcome.html");

                                rd.forward(request, response);


                        }else{
                                RequestDispatcher rd =
getServletContext().getRequestDispatcher("/login.html");
                                PrintWriter out= response.getWriter();

                                out.println("<font color=red>No user found with given email
id, please register first.</font>");
                                rd.include(request, response);
                        }
                } catch (SQLException | ClassNotFoundException e) {
                        e.printStackTrace();

                        throw new ServletException("DB Connection problem.");
                }finally{
                        try {
                                rs.close();
```

```
                        ps.close();
                    } catch (SQLException e) {
                    }

                }
                }
        }

}
```

RegisterServlet.java:

The RegisterServlet also has the doPost() method overridden. Refer to the Register.html file for the method attribute of the html form element for this. The operations performed here is to capture the details of a new user that needs to be registered with the application.

```
package com.tcs.ilp.servlet;

import java.io.IOException;
import java.io.PrintWriter;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

publicclass RegisterServlet extends HttpServlet {

        /**
         *
         */
        private static final long serialVersionUID = 1L;

        protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
                String uname = request.getParameter("username");
                String password = request.getParameter("password");
```

```
                String name = request.getParameter("name");
                String country = request.getParameter("country");
                String errorMsg = null;
                if(uname == null || uname.equals("")){
                        errorMsg = "User Name can't be null or empty.";
                }
                if(password == null || password.equals("")){
                        errorMsg = "Password can't be null or empty.";
                }
                if(name == null || name.equals("")){
                        errorMsg = "Name can't be null or empty.";
                }
                if(country == null || country.equals("")){
                        errorMsg = "Country can't be null or empty.";
                }

                if(errorMsg != null){
                        RequestDispatcher rd =
getServletContext().getRequestDispatcher("/register.html");
                        PrintWriter out= response.getWriter();
                        out.println("<font color=red>"+errorMsg+"</font>");
                        rd.include(request, response);
                }else{

                String serverName = "172.24.137.30";
                String portNumber = "1521";
                String sid = "orcl";
                String url = "jdbc:oracle:thin:@" + serverName + ":" + portNumber + ":" +
sid;
                String dbUsername =  "e137852";
                String dbPassword = "cJRxepdfp";
                PreparedStatement ps = null;
                Connection conn =null;

                try {
// load the JDBC-ODBC Bridge driver
                        Class.forName("oracle.jdbc.driver.OracleDriver");

                        // connect to db using DriverManager
                        System.out.println("The Jdbc URL is "+url);
                        conn =DriverManager.getConnection( url, dbUsername, dbPassword );
                        ps = conn.prepareStatement("insert into Users(name,username,country,
password) values (?,?,?,?)");
                        ps.setString(1, name);
                        ps.setString(2, uname);
                        ps.setString(3, country);
                        ps.setString(4, password);

                        ps.execute();
```

```
                    //forward to login page to login
                    RequestDispatcher rd =
getServletContext().getRequestDispatcher("/Login.html");
                    PrintWriter out= response.getWriter();
                    out.println("<font color=green>Registration successful, please login
below.</font>");
                    rd.include(request, response);
             } catch (SQLException | ClassNotFoundException e) {
                    e.printStackTrace();

                    throw new ServletException("DB Connection problem.");
             }finally{
                    try {

                           ps.close();
                    } catch (SQLException e) {
                    }

             }
             }
       }

}
```

In both these servlets, we see that the database operations of data retrieval and data insertion are handled within the servlet. All database operations are initiated from the beginning i.e. from making the DB connection with the use of relevant drivers, DB credentials and processed till conclusion. What do we observe as major disadvantages here?

5.4.3 Disadvantages of JDBC in Servlets

The major disadvantages of using the above approach are

- Code redundancy – Database access mechanism and the logic have been repeated in both the servlets
- The database access parameters are exposed and not abstract
- Poor readability and less maintainable as it lacks the modularity.
- Business logic (flow of application control) and database operations are mingled and reside in the same methods

5.4.4 Introducing the Data Access Objects

Data Access Objects (DAO) is those that provide an abstraction mechanism to operate on databases or any persistence logic. The logic is to separate all the database access mechanism from the control or business logic of any application. These DAO objects are built as classes in Java that suffice the database access process with utility methods to perform the database operations. So let us get down to segregate the database operations of both our servlets into a single DAO object

Since the application works on insertion and retrieval of user information, let us call this DAO class as UserDAO.java. The following is how the UserDAO.java will be organized.

```
package com.tcs.ilp.dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class UserDAO {

	static String serverName = "172.24.137.30";
	static String portNumber = "1521";
	static String sid = "orcl";
	static String url = "jdbc:oracle:thin:@" + serverName + ":" + portNumber
			+ ":" + sid;
	static String dbUsername = "e137852";
	static String dbPassword = "cJRxepdfp";

	public static boolean insertUser(String uname, String password,
			String name, String country) {
		boolean status = false;

		PreparedStatement ps = null;
		Connection conn = null;

		try {
			// load the JDBC-ODBC Bridge driver
			Class.forName("oracle.jdbc.driver.OracleDriver");

			// connect to db using DriverManager
			System.out.println("The Jdbc URL is " + url);
```

```
                    conn = DriverManager.getConnection(url, dbUsername, dbPassword);
                    ps = conn
                                 .prepareStatement("insert into
Users(name,username,country, password) values (?,?,?,?)");
                    ps.setString(1, name);
                    ps.setString(2, uname);
                    ps.setString(3, country);
                    ps.setString(4, password);

                    status = ps.execute();

             } catch (SQLException | ClassNotFoundException e) {
                    e.printStackTrace();

             } finally {
                    try {

                           ps.close();
                    } catch (SQLException e) {
                    }

             }

             return status;
       }

       public static boolean getUser(String uname, String password) {

             boolean userFound = false;

             PreparedStatement ps = null;
             Connection conn = null;
             ResultSet rs = null;

             try {
                    // load the JDBC-ODBC Bridge driver
                    Class.forName("oracle.jdbc.driver.OracleDriver");

                    // connect to db using DriverManager
                    System.out.println("The Jdbc URL is " + url);
                    conn = DriverManager.getConnection(url, dbUsername, dbPassword);
                    ps = conn
                                 .prepareStatement("select name, username ,country
from Users where username=? and password=? limit 1");
                    ps.setString(1, uname);
                    ps.setString(2, password);
                    rs = ps.executeQuery();

                    if (rs != null && rs.next()) {
                           userFound = true;
```

```
                        } else {
                                userFound = false;
                        }
                } catch (SQLException | ClassNotFoundException e) {
                        e.printStackTrace();
                } finally {
                        try {
                                rs.close();
                                ps.close();
                        } catch (SQLException e) {
                        }

                }
                return userFound;
        }

}
```

Now that we have an UserDAO object, with the two methods to perform the tasks involved in the scenario that is being discussed, let us now modify our servlets to call these methods to perform the insert and select db operations

LoginServlet.java

```
package com.tcs.ilp.servlet;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.tcs.ilp.dao.UserDAO;

public class LoginServlet extends HttpServlet {

        /**
         *
         */
        private static final long serialVersionUID = 1L;

        protected void doPost(HttpServletRequest request,
                        HttpServletResponse response) throws ServletException, IOException {
```

```
            String uname = request.getParameter("username");
            String password = request.getParameter("password");
            String errorMsg = null;
            if (uname == null || uname.equals("")) {
                  errorMsg = "User Name can't be null or empty";
            }
            if (password == null || password.equals("")) {
                  errorMsg = "Password can't be null or empty";
            }

            if (errorMsg != null) {
                  RequestDispatcher rd = getServletContext().getRequestDispatcher(
                                 "/login.html");
                  PrintWriter out = response.getWriter();
                  out.println("<font color=red>" + errorMsg + "</font>");
                  rd.include(request, response);
            } else {

                  boolean userFound = UserDAO.getUser(uname, password);

                  if (userFound) {
                        // Print welcome message to user
                        RequestDispatcher rd = getServletContext()
                                    .getRequestDispatcher("/Welcome.html");

                        rd.forward(request, response);

                  } else {
                        RequestDispatcher rd = getServletContext()
                                    .getRequestDispatcher("/login.html");
                        PrintWriter out = response.getWriter();

                        out.println("<font color=red>No user found with given email id,
please register first.</font>");
                        rd.include(request, response);
                  }
            }

      }
}
```

RegisterServlet.java

```
package com.tcs.ilp.servlet;

import java.io.IOException;
```

```java
import java.io.PrintWriter;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.tcs.ilp.dao.UserDAO;

public class RegisterServlet extends HttpServlet {

	/**
	 *
	 */
	private static final long serialVersionUID = 1L;

	protected void doPost(HttpServletRequest request,
			HttpServletResponse response) throws ServletException, IOException {
		String uname = request.getParameter("username");
		String password = request.getParameter("password");
		String name = request.getParameter("name");
		String country = request.getParameter("country");
		String errorMsg = null;
		if (uname == null || uname.equals("")) {
			errorMsg = "User Name can't be null or empty.";
		}
		if (password == null || password.equals("")) {
			errorMsg = "Password can't be null or empty.";
		}
		if (name == null || name.equals("")) {
			errorMsg = "Name can't be null or empty.";
		}
		if (country == null || country.equals("")) {
			errorMsg = "Country can't be null or empty.";
		}

		if (errorMsg != null) {
			RequestDispatcher rd = getServletContext().getRequestDispatcher(
				"/register.html");
			PrintWriter out = response.getWriter();
			out.println("<font color=red>" + errorMsg + "</font>");
			rd.include(request, response);
		} else {

			if (UserDAO.insertUser(uname, password, name, country)) {

				// forward to login page to login
				RequestDispatcher rd = getServletContext()
					.getRequestDispatcher("/Login.html");
```

```
                        PrintWriter out = response.getWriter();
                        out.println("<font color=green>Registration successful, please login
below.</font>");
                        rd.include(request, response);
                    }

            }
        }

}
```

It's now made even simpler with a data layer that clearly isolated the database tasks. The servlet is the controller that interfaces with the user interface and the data layer. But we still see some redundant code in our UserDAO.java. The information needed to get a Connection and process of getting a connection has been duplicated in both the methods, namely, insertUser() and getUser(). Also, considering the size of the current application and our database design, all the details of a new User is accepted as arguments by the data layer method. But if the details are too large and needs to be inserted to multiple tables, then this approach gets more complex too.

To mitigate such situations and also to keep the core information of the application in line with the database structure, we create simple POJO classes that are imaged with the database structure. So let us create a UserBean.java with the necessary information to be transacted.

UserBean.java

```
package com.tcs.ilp.bean;

public class UserBean {

        private String username = null;
        private String password = null;
        private String name = null;
        private String country = null;

        public String getUsername() {
                return username;
        }
        public void setUsername(String username) {
                this.username = username;
        }
        public String getPassword() {
```

```
		return password;
	}
	public void setPassword(String password) {
		this.password = password;
	}
	public String getName() {
		return name;
	}
	public void setName(String name) {
		this.name = name;
	}
	public String getCountry() {
		return country;
	}
	public void setCountry(String country) {
		this.country = country;
	}

}
```

The below ConnectionManager.java class will handle the code reusability and providing connections for any DAO.

ConnectionManager.java

```
package com.tcs.ilp.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConnectionManager {

	static final String serverName = "172.24.137.30";
	static final String portNumber = "1521";
	static final String sid = "orcl";
	static final String url = "jdbc:oracle:thin:@" + serverName + ":" + portNumber
			+ ":" + sid;
	static final String dbUsername = "e137852";
	static final String dbPassword = "cJRxepdfp";

	public static Connection getConnection (){

		Connection conn = null;
```

```
            try {
                  // load the JDBC-ODBC Bridge driver
                  Class.forName("oracle.jdbc.driver.OracleDriver");

                  // connect to db using DriverManager
                  System.out.println("The Jdbc URL is " + url);
                  conn = DriverManager.getConnection(url, dbUsername, dbPassword);
            } catch (ClassNotFoundException | SQLException e){
                  e.printStackTrace();
            }
            return conn;


      }

}
```

Both servlets are now changed as the below ones

LoginServlet.java

```
package com.tcs.ilp.servlet;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.tcs.ilp.bean.UserBean;
import com.tcs.ilp.dao.UserDAO;

public class LoginServlet extends HttpServlet {

      /**
       *
       */
      private static final long serialVersionUID = 1L;

      protected void doPost(HttpServletRequest request,
                  HttpServletResponse response) throws ServletException, IOException {
```

```
            String uname = request.getParameter("username");
            String password = request.getParameter("password");
            String errorMsg = null;
            if (uname == null || uname.equals("")) {
                  errorMsg = "User Name can't be null or empty";
            }
            if (password == null || password.equals("")) {
                  errorMsg = "Password can't be null or empty";
            }

            if (errorMsg != null) {
                  RequestDispatcher rd = getServletContext().getRequestDispatcher(
                                 "/login.html");
                  PrintWriter out = response.getWriter();
                  out.println("<font color=red>" + errorMsg + "</font>");
                  rd.include(request, response);
            } else {

                  UserBean userBn = new UserBean ();
                  userBn.setUsername(uname);
                  userBn.setPassword(password);
                  boolean userFound = UserDAO.getUser(userBn);

                  if (userFound) {
                        // Print welcome message to user
                        RequestDispatcher rd = getServletContext()
                                    .getRequestDispatcher("/Welcome.html");

                        rd.forward(request, response);

                  } else {
                        RequestDispatcher rd = getServletContext()
                                    .getRequestDispatcher("/login.html");
                        PrintWriter out = response.getWriter();

                        out.println("<font color=red>No user found with given email id,
please register first.</font>");
                        rd.include(request, response);
                  }
            }

      }
}
```

RegisterServlet.java

```
package com.tcs.ilp.servlet;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.tcs.ilp.bean.UserBean;
import com.tcs.ilp.dao.UserDAO;

public class RegisterServlet extends HttpServlet {

	/**
	 *
	 */
	private static final long serialVersionUID = 1L;

	protected void doPost(HttpServletRequest request,
			HttpServletResponse response) throws ServletException, IOException {
		String uname = request.getParameter("username");
		String password = request.getParameter("password");
		String name = request.getParameter("name");
		String country = request.getParameter("country");
		String errorMsg = null;
		if (uname == null || uname.equals("")) {
			errorMsg = "User Name can't be null or empty.";
		}
		if (password == null || password.equals("")) {
			errorMsg = "Password can't be null or empty.";
		}
		if (name == null || name.equals("")) {
			errorMsg = "Name can't be null or empty.";
		}
		if (country == null || country.equals("")) {
			errorMsg = "Country can't be null or empty.";
		}

		if (errorMsg != null) {
			RequestDispatcher rd = getServletContext().getRequestDispatcher(
				"/register.html");
			PrintWriter out = response.getWriter();
			out.println("<font color=red>" + errorMsg + "</font>");
			rd.include(request, response);
```

```
            } else {

                    UserBean userBn = new UserBean();
                    userBn.setName(name);
                    userBn.setCountry(country);
                    userBn.setUsername(uname);
                    userBn.setPassword(password);

                    if (UserDAO.insertUser(userBn)) {

                            // forward to login page to login
                            RequestDispatcher rd = getServletContext()
                                          .getRequestDispatcher("/Login.html");
                            PrintWriter out = response.getWriter();
                            out.println("<font color=green>Registration successful, please login
below.</font>");
                            rd.include(request, response);
                    }

            }
        }

}
```

5.4.5 Advantages of Data layers

Let us now list down the advantages as seen through the example in this document

- Better readability and maintainability of the developed code
- Database processes abstraction
- Removal of code redundancy and reusability of code
- Use of POJOs as the data structure for simplification of the database operations

5.4.6 Service Layers

Service Layers are intermediate layers in Java EE application to implement code reusability to its maximum. It also acts as a Transfer Object and Mediating layer in the cases of Java EE applications. For very large scale applications, the service layers are points of performing certain business logic or rules validation before the data is processed and committed to the DB. Since our example just deals about a simple user operation, we will use our service layer as a Transfer Object layer.

Let us build a UserService.java containing the service methods for each of my DB operation. The servlet implementations will now invoke the service methods in place of the DAO methods.

UserService.java

```
package com.tcs.ilp.service;

import com.tcs.ilp.bean.UserBean;
import com.tcs.ilp.dao.UserDAO;

public class UserService {

	UserDAO userDAO = new UserDAO();

	public boolean insertUserService (UserBean userBn) {

		boolean status = userDAO.insertUser(userBn);

		return status;
	}

	public boolean getUserService (UserBean userBn){

		boolean userFound = userDAO.getUser(userBn);

		return userFound;
	}

}
```

LoginServlet.java

```
package com.tcs.ilp.servlet;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.tcs.ilp.bean.UserBean;
import com.tcs.ilp.service.UserService;

public class LoginServlet extends HttpServlet {

	/**
	 *
	 */
	private static final long serialVersionUID = 1L;

	protected void doPost(HttpServletRequest request,
			HttpServletResponse response) throws ServletException, IOException {
		String uname = request.getParameter("username");
		String password = request.getParameter("password");
		String errorMsg = null;
		if (uname == null || uname.equals("")) {
			errorMsg = "User Name can't be null or empty";
		}
		if (password == null || password.equals("")) {
			errorMsg = "Password can't be null or empty";
		}

		if (errorMsg != null) {
			RequestDispatcher rd = getServletContext().getRequestDispatcher(
				"/login.html");
			PrintWriter out = response.getWriter();
			out.println("<font color=red>" + errorMsg + "</font>");
			rd.include(request, response);
		} else {

			UserBean userBn = new UserBean ();
			userBn.setUsername(uname);
			userBn.setPassword(password);
			UserService userSvc = new UserService();
			boolean userFound = userSvc.getUserService(userBn);
```

```
                    if (userFound) {
                          // Print welcome message to user
                          RequestDispatcher rd = getServletContext()
                                        .getRequestDispatcher("/Welcome.html");

                          rd.forward(request, response);

                    } else {
                          RequestDispatcher rd = getServletContext()
                                        .getRequestDispatcher("/login.html");
                          PrintWriter out = response.getWriter();

                          out.println("<font color=red>No user found with given email id,
please register first.</font>");
                          rd.include(request, response);
                    }
             }

       }
}
```

RegisterServlet.java:

```
package com.tcs.ilp.servlet;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import com.tcs.ilp.bean.UserBean;
import com.tcs.ilp.service.UserService;

public class RegisterServlet extends HttpServlet {


       private static final long serialVersionUID = 1L;

       protected void doPost(HttpServletRequest request,
                    HttpServletResponse response) throws ServletException, IOException {
             String uname = request.getParameter("username");
             String password = request.getParameter("password");
             String name = request.getParameter("name");
             String country = request.getParameter("country");
             String errorMsg = null;
```

```
            if (uname == null || uname.equals("")) {
                  errorMsg = "User Name can't be null or empty.";
            }
            if (password == null || password.equals("")) {
                  errorMsg = "Password can't be null or empty.";
            }
            if (name == null || name.equals("")) {
                  errorMsg = "Name can't be null or empty.";
            }
            if (country == null || country.equals("")) {
                  errorMsg = "Country can't be null or empty.";
            }

            if (errorMsg != null) {
                  RequestDispatcher rd = getServletContext().getRequestDispatcher(
                                 "/register.html");
                  PrintWriter out = response.getWriter();
                  out.println("<font color=red>" + errorMsg + "</font>");
                  rd.include(request, response);
            } else {

                  UserBean userBn = new UserBean();
                  userBn.setName(name);
                  userBn.setCountry(country);
                  userBn.setUsername(uname);
                  userBn.setPassword(password);

                  UserService userSvc = new UserService();

                  if (userSvc.insertUserService(userBn)) {

                        // forward to login page to login
                        RequestDispatcher rd = getServletContext()
                                    .getRequestDispatcher("/Login.html");
                        PrintWriter out = response.getWriter();
                        out.println("<font color=green>Registration successful, please login
below.</font>");
                        rd.include(request, response);
                  }

            }
      }

}
```

Note: The DAO methods are to be made non-static as service creates the objects to invoke them.

5.4.6 Summary

- Understanding of the JDBC implementation in servlets
- Creating data layers and abstraction of database tasks
- Creating service layers for transferring objects